



When Should Language Models Search?

A Topology-Aware Analysis of Reasoning Strategies

M.Tech Thesis Defense

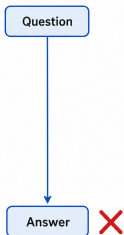
Md Tanzeel Adam Khan (2411AI67)

Department of Computer Science and Engineering
Supervisor: Dr. Asif Ekbal

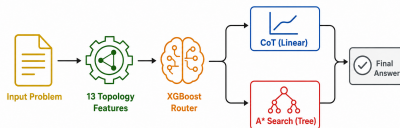
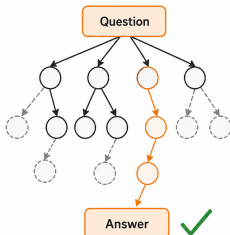
June 2026

Example: “*Is shrimp scampi free of plastic?*” — CoT ✗: 5 steps, misses microplastics (wrong).
A* ✓: finds key fact in step 1 (correct). **When is search worth the cost?**

Chain-of-Thought (Linear)

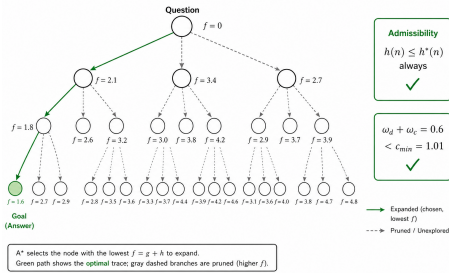


A* Search (Branching)



- **Our A* implementation**
- 13 topology features (input only)
- XGBoost routes *before* generation
- Zero LLM cost for routing

A* Search Over Reasoning Traces



Why A* (not ToT, not MCTS)?

- Isolates **branching** from evaluator design
- Clean separation: frontier + cost + heuristic
- ToT entangles evaluator; MCTS adds rollout noise

Formulation:

$$f(n) = g(n) + h(n)$$

$$g(n) = d_n + \sum (1 - c_t) \quad (\text{length} + \text{uncertainty})$$

$$h(n) = \omega_d(H_{\min} - d_n) + \omega_c(1 - E(n))$$

Admissibility (proven):

- $h(n) \leq h^*(n)$ for all non-goal nodes
- Condition: $\omega_d + \omega_c \leq c_{\min}$
- Our values: $0.3 + 0.3 = 0.6 < 1.01$ ✓
- Guarantees optimal path under budget**

Route to A* (search)

High branching coefficient
Multiple hops / entities
Many connectives (if, because)
Low linearity score

Route to CoT (linear)

Low branching coefficient
Short, single-step question
Direct factual recall
High linearity score

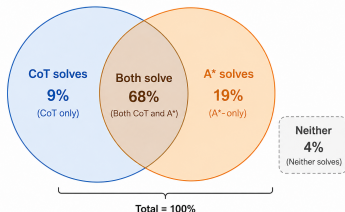
- 1 Question length
- 2 Context length
- 3 Logical connectives *
- 4 Negation markers
- 5 Multi-hop indicators *
- 6 Comparison words
- 7 Sub-question marks

- 8 Context sentences *
- 9 Named entities
- 10 Linearity score
- 11 Answer options
- 12 Depth estimate
- 13 Branching complexity *

$$BC = \frac{ctx_sents \times hop}{q_len} \quad (R^2 = 0.94)$$

* = top XGBoost importance. All features from input text only — zero LLM inference.

Complementarity Between CoT and A*



Instance-level (8B):

Dataset	CoT-only	A*-only
StrategyQA	14.3%	10.8%
HotpotQA	17.2%	17.4%
CodeContests	9.2%	18.8%

Oracle gap: up to **14pp**

How the router leverages this:

- Each instance: features predict $\rightarrow$ CoT or A*
- "Branching" problems $\rightarrow$ A* (multi-hop, algorithmic)
- "Linear" problems $\rightarrow$ CoT (factual, single-step)
- Router recovers **40%** of the oracle gap

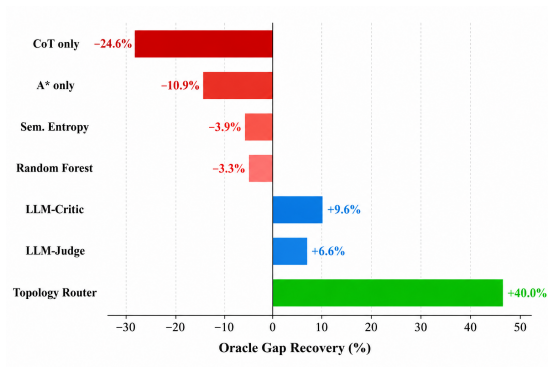
Practical benefit:

- A* costs $24 \times$ CoT
- Router applies A* only when needed ($\sim 30\%$ of instances)
- Effective cost: $\sim 8 \times$ CoT (not $24 \times$)
- Best of both worlds:** high accuracy, manageable cost

Neither method alone is optimal $\rightarrow$ topology routing fills the gap

Dataset	LLaMA-8B		Qwen-0.5B		Qwen-14B	
	CoT	A*	CoT	A*	CoT	A*
StrategyQA	64.5	74.8	53.6	53.4	76.0	75.6
LogicQA	45.8	44.9	24.0	20.2	57.9	58.1
HotpotQA	76.8	80.1	8.4	30.0	60.2	83.3
GSM8K	85.0	86.1	26.8	14.5	61.6	80.6
MATH	78.0	79.1	32.8	16.2	44.0	47.2
HumanEval	67.5	64.4	33.7	20.2	77.9	76.1
MBPP	39.8	36.8	24.0	20.8	46.6	45.0
CodeContests	77.4	87.0	49.8	68.2	99.6	94.4

- **Neither dominates.** Red = A* wins; Blue = CoT wins
- A* **+19pp** GSM8K (14B); **+18.4pp** CodeContests (0.5B)
- 8 benchmarks $\times$ 3 scales $\times$ 4 methods (CoT, A*, SC, ToT)



Why it works:

- 38% fluent traces = *wrong*
- Judges fooled by readability
- Our router: **structure only**

Key numbers:

- +40% gap recovery overall
- 95% on CodeContests
- 3 domains: QA + Math + Code
- Zero routing cost

Route by structure, not trace quality



Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in LLMs. *NeurIPS*.



Yao, S. et al. (2024). Tree of thoughts: Deliberate problem solving with LLMs. *NeurIPS*.



Hao, S. et al. (2023). Reasoning with language model is planning with world model. *EMNLP*.



Wang, X. et al. (2023). Self-consistency improves chain of thought reasoning. *ICLR*.



Snell, C. et al. (2024). Scaling LLM test-time compute optimally. *arXiv:2408.03314*.



Hart, P. et al. (1968). A formal basis for heuristic determination of minimum cost paths. *IEEE Trans*.



Chen, M. et al. (2021). Evaluating large language models trained on code. *arXiv:2107.03374*.



Kossen, J. et al. (2024). Semantic entropy probes. *arXiv:2406.15927*.



Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench. *NeurIPS*.



Madaan, A. et al. (2023). Self-Refine: Iterative refinement with self-feedback. *NeurIPS*.

Paper submitted to ACL Rolling Review (ARR)

Thank You